



RELATÓRIO — BIG DATA E NoSQL

Thiago Dellano Machado de Oliveira | 24101272

1. Introdução

O presente trabalho tem como objetivo explorar conceitos fundamentais de Big Data e bancos de dados NoSQL por meio de atividades práticas. Durante o desenvolvimento, foram utilizadas ferramentas amplamente empregadas no mercado, como MongoDB, Docker, Python e Pandas, permitindo a simulação de um ambiente real de engenharia de dados.

2. Configuração do Ambiente

Inicialmente, foi realizada a configuração do ambiente utilizando o Docker para a execução do MongoDB em um container. Essa abordagem permite maior portabilidade e facilidade na replicação do ambiente.

O container foi iniciado com sucesso, garantindo a disponibilidade do banco de dados na porta padrão 27017.

3. MongoDB Local (On-Premises)

Foi utilizado o MongoDB local para a criação e manipulação de dados. As seguintes operações foram realizadas:

- Criação do banco de dados [escola](#)
- Criação da coleção [alunos](#)
- Inserção de documentos utilizando [insertOne](#)

- Consulta de dados com [find\(\)](#)

Exemplo de documento inserido:

```
{  
  "nome": "Thiago",  
  "idade": 21,  
  "curso": "Software"  
}
```

Essas operações demonstraram o funcionamento básico de um banco de dados NoSQL orientado a documentos.

4. Integração com Python (PyMongo)

Foi realizada a integração entre o MongoDB e a linguagem Python utilizando a biblioteca PyMongo. A conexão foi estabelecida com o banco local, permitindo a leitura dos dados diretamente no ambiente Jupyter Notebook.

Essa etapa evidenciou a capacidade de interação entre aplicações e bancos NoSQL.

5. Manipulação de Dados com Pandas

Utilizando a biblioteca Pandas, foi criado um DataFrame contendo dados simulados de alunos:

- Nome
- Idade
- Curso

O DataFrame foi utilizado como base para simular um conjunto de dados estruturados, semelhante aos encontrados em plataformas como Kaggle.

6. Ingestão de Dados no MongoDB

Os dados do DataFrame foram convertidos para formato JSON e inseridos no MongoDB utilizando o método [insert many](#).

Essa etapa representa um processo de ingestão de dados, comum em pipelines de Big Data, onde dados externos são carregados para um banco de dados para posterior análise.

7. Consulta e Validação dos Dados

Após a inserção, foi realizada uma nova consulta no banco de dados, confirmando a presença de múltiplos registros.

Essa validação garantiu que os dados foram armazenados corretamente.

8. Análise de Dados (Censo IES)

Foi realizada uma análise exploratória de dados simulando informações do Censo da Educação Superior.

As seguintes análises foram aplicadas:

- Média de alunos: 25.600
- Distribuição por estado: DF, SP e RJ
- Agrupamento por instituição: soma total de alunos por

Essas operações foram realizadas utilizando a biblioteca Pandas, demonstrando a capacidade de extrair informações relevantes a partir dos dados.

9. Conclusão

O desenvolvimento deste trabalho permitiu a aplicação prática de conceitos de Big Data e NoSQL, integrando diferentes tecnologias como MongoDB, Docker, Python e Pandas.

A utilização dessas ferramentas possibilitou a criação de um fluxo completo de dados, desde a inserção até a análise, evidenciando a importância dessas tecnologias no contexto atual da engenharia de dados.

Além disso, a abordagem prática contribuiu significativamente para a compreensão dos conceitos estudados, aproximando o ambiente acadêmico da realidade do mercado.

10. Prints

```
1 de abr 08:45
thiggas@thiggas-VirtualBox: ~

thiggas@thiggas-VirtualBox:~$ sudo apt update
[sudo] senha para thiggas:
Atingido:1 https://packages.microsoft.com/repos/code stable InRelease
Atingido:2 http://archive.ubuntu.com/ubuntu noble InRelease
Atingido:3 http://security.ubuntu.com/ubuntu noble-security InRelease
Atingido:4 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Atingido:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Lendo listas de pacotes... Pronto
Construindo árvore de dependências... Pronto
Lendo informação de estado... Pronto
83 pacotes podem ser atualizados. Execute 'apt list --upgradable' para vê-los.
thiggas@thiggas-VirtualBox:~$ sudo apt install mongodb
Lendo listas de pacotes... Pronto
Construindo árvore de dependências... Pronto
Lendo informação de estado... Pronto
O pacote mongodb não está disponível, mas é referenciado por outro pacote.
Isto pode significar que o pacote está faltando, ficou obsoleto ou
está disponível somente a partir de outra fonte

E: O pacote 'mongodb' não tem candidato para instalação
thiggas@thiggas-VirtualBox:~$ mongosh
mongosh: comando não encontrado
thiggas@thiggas-VirtualBox:~$ ^C
thiggas@thiggas-VirtualBox:~$ docker run -d -p 27017:27017 --name mongodb mongo
Comando 'docker' não encontrado, mas poder ser instalado com:
sudo snap install docker          # version 28.4.0, or
sudo apt install docker.io        # version 28.2.2-0ubuntu1-24.04.1
sudo apt install podman-docker    # version 4.9.3+ds1-1ubuntu0.2
Veja 'snap info docker' para versões adicionais.
thiggas@thiggas-VirtualBox:~$ docker ps
Comando 'docker' não encontrado, mas poder ser instalado com:
sudo snap install docker          # version 28.4.0, or
sudo apt install docker.io        # version 28.2.2-0ubuntu1-24.04.1
sudo apt install podman-docker    # version 4.9.3+ds1-1ubuntu0.2
Veja 'snap info docker' para versões adicionais.
thiggas@thiggas-VirtualBox:~$ sudo apt install docker.io -y
Lendo listas de pacotes... Pronto
Construindo árvore de dependências... Pronto
Lendo informação de estado... Pronto
Os pacotes adicionais a seguir serão instalados:
  bridge-utils containerd pigz runc ubuntu-fan
Pacotes sugeridos:
  ifupdown aufs-tools btrfs-progs cgroupfs-mount | cgroup-lite debootstrap
  docker-buildx docker-compose-v2 docker-doc rinse zfs-fuse | zfsutils
Os NOVOS pacotes a seguir serão instalados:
  bridge-utils containerd docker.io pigz runc ubuntu-fan
```

Figura 1 – Atualização do sistema e tentativa de instalação do MongoDB

```
thiggas@thiggas-VirtualBox:~$ sudo systemctl start docket
Failed to start docket.service: Unit docket.service not found.
thiggas@thiggas-VirtualBox:~$ sudo systemctl enable docker
thiggas@thiggas-VirtualBox:~$ sudo systemctl start docket
Failed to start docket.service: Unit docket.service not found.
thiggas@thiggas-VirtualBox:~$ sudo usermod -aG docker thiggas
thiggas@thiggas-VirtualBox:~$ docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1-24.04.1
thiggas@thiggas-VirtualBox:~$ docker run -d -p 27017:27017 --name mongodb mongo
docker: permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Head "http://%2Fvar%2Frun%2Fdocker.sock/_ping": dial unix /var/run/docke
r.sock: connect: permission denied

Run 'docker run --help' for more information
thiggas@thiggas-VirtualBox:~$ sudo docker run -d -p 27017:27017 --name mongodb mongo
Unable to find image 'mongo:latest' locally
latest: Pulling from library/mongo
817807f3c64e: Pull complete
43c433b73ba8: Pull complete
13335f999508: Pull complete
389b5cf90dda: Pull complete
e5522a7abaf1: Pull complete
e2a40f03c0fa: Pull complete
5b10863adf74: Pull complete
1d001d987e73: Pull complete
Digest: sha256:eea8506335198f8b359865b32004036310854a935fbd317083817c614152818f
Status: Downloaded newer image for mongo:latest
d38ba3d6194a20b674030bfb38592428605092b312fef7da0fb51de8a7b4b256
thiggas@thiggas-VirtualBox:~$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
d38ba3d6194a   mongo    "docker-entrypoint.s..." 39 seconds ago Up 38 seconds 0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp   mongodb
thiggas@thiggas-VirtualBox:~$ sudo docker exec -it mongodb mongosh
Current Mongosh Log ID: 69cd043bf950e3927a44ba88
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.2
Using MongoDB:      8.2.6
Using Mongosh:       2.8.2

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2026-04-01T11:38:06.214+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
2026-04-01T11:38:06.224+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted.
```

Figura 2 – Instalação do Docker e execução do container
MongoDB

```
1 de abr 00:45
Captura de tela Agora mesmo
Captura de tela obtida
Você pode colar a imagem a partir da área de transferência.

1d801d987e73: Pull complete
Digest: sha256:eea8506335198f8b359865b32004036310854a935fbd31708381
Status: Downloaded newer image for mongo:latest
d38ba3d6194a20b674030bf38592428605092b312fef7da0fb51de8a7b4b256
thiggas@thiggas-VirtualBox:~$ sudo docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
d38ba3d6194a   mongo     "docker-entrypoint.s..." 39 seconds ago Up 38 seconds 0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp   mongodb
thiggas@thiggas-VirtualBox:~$ sudo docker exec -it mongodb mongosh
Current Mongosh Log ID: 69cd043bf950e3927a44ba88
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.2
Using MongoDB:      8.2.6
Using Mongosh:      2.8.2

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2026-04-01T11:38:06.214+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
2026-04-01T11:38:06.394+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2026-04-01T11:38:06.394+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-04-01T11:38:06.394+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-04-01T11:38:06.394+00:00: We suggest setting the contents of sysfsFile to 0.
2026-04-01T11:38:06.394+00:00: We suggest setting swappiness to 0 or 1, as swapping can cause performance problems.
-----

test> use escola
|
| db.alunos.insertOne({
|   nome: "Thiago",
|   idade: 21,
|   curso: "Software"
| })
|
| db.alunos.find()
switched to db escola
escola>
(To exit, press Ctrl+C again or Ctrl+D or type .exit)
escola>

thiggas@thiggas-VirtualBox:~$ script sessao.txt
Script iniciado, arquivo de registro de saída é "sessao.txt".
thiggas@thiggas-VirtualBox:~$
```

Figura 3 – Execução do MongoDB e inserção de dados via terminal

The screenshot shows a Jupyter Notebook running in a web browser. The notebook is titled "Untitled" and has a last checkpoint 10 minutes ago. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and code execution. The code is written in Python and interacts with a database.

```
# acessar banco
db = client["escola"]

# acessar coleção
colecão = db["alunos"]

# mostrar dados
for aluno in coleção.find():
    print(aluno)

{'_id': ObjectId('69cfcb2f2a83fd22c244ba89'), 'nome': 'Thiago', 'idade': 21, 'curso': 'Software'}
```

[3]: `import pandas as pd`

```
# criar um dataset simples (caso não tenha Kaggle)
dados = {
    "nome": ["Ana", "Carlos", "Julia"],
    "idade": [22, 30, 25],
    "curso": ["ADS", "SI", "CC"]
}

df = pd.DataFrame(dados)
df
```

	nome	idade	curso
0	Ana	22	ADS
1	Carlos	30	SI
2	Julia	25	CC

```
[4]: coleção.insert_many(df.to_dict("records"))

[4]: InsertManyResult([ObjectId('69cf08d61504b13ee14932d'), ObjectId('69cf08d61504b13ee14932e'), ObjectId('69cf08d61504b13ee14932f')], acknowledged=True)

[5]: for aluno in coleção.find():
    print(aluno)

{'_id': ObjectId('69cfcb2f2a83fd22c244ba89'), 'nome': 'Thiago', 'idade': 21, 'curso': 'Software'}
{'_id': ObjectId('69cf08d61504b13ee14932d'), 'nome': 'Ana', 'idade': 22, 'curso': 'ADS'}
{'_id': ObjectId('69cf08d61504b13ee14932e'), 'nome': 'Carlos', 'idade': 30, 'curso': 'SI'}
{'_id': ObjectId('69cf08d61504b13ee14932f'), 'nome': 'Julia', 'idade': 25, 'curso': 'CC'}
```

Figura 4 – Integração inicial com Python (Jupyter Notebook)

The screenshot shows a JupyterLab interface with a notebook titled 'Untitled'. The notebook contains the following code:

```
[1]: from pymongo import MongoClient

# conectar no MongoDB (Docker)
client = MongoClient("mongodb://localhost:27017/")

# acessar banco
db = client["escola"]

# acessar coleção
colecacao = db["alunos"]

# mostrar dados
for aluno in colecacao.find():
    print(aluno)

{'_id': ObjectId('69cfcb2f2a83fd22c244ba89'), 'nome': 'Thiago', 'idade': 21, 'curso': 'Software'}
```

```
[3]: import pandas as pd

# criar um dataset simples (caso não tenha Kaggle)
dados = {
    "nome": ["Ana", "Carlos", "Julia"],
    "idade": [22, 30, 25],
    "curso": ["ADS", "SI", "CC"]
}

df = pd.DataFrame(dados)
df
```

The output of the third cell is a DataFrame:

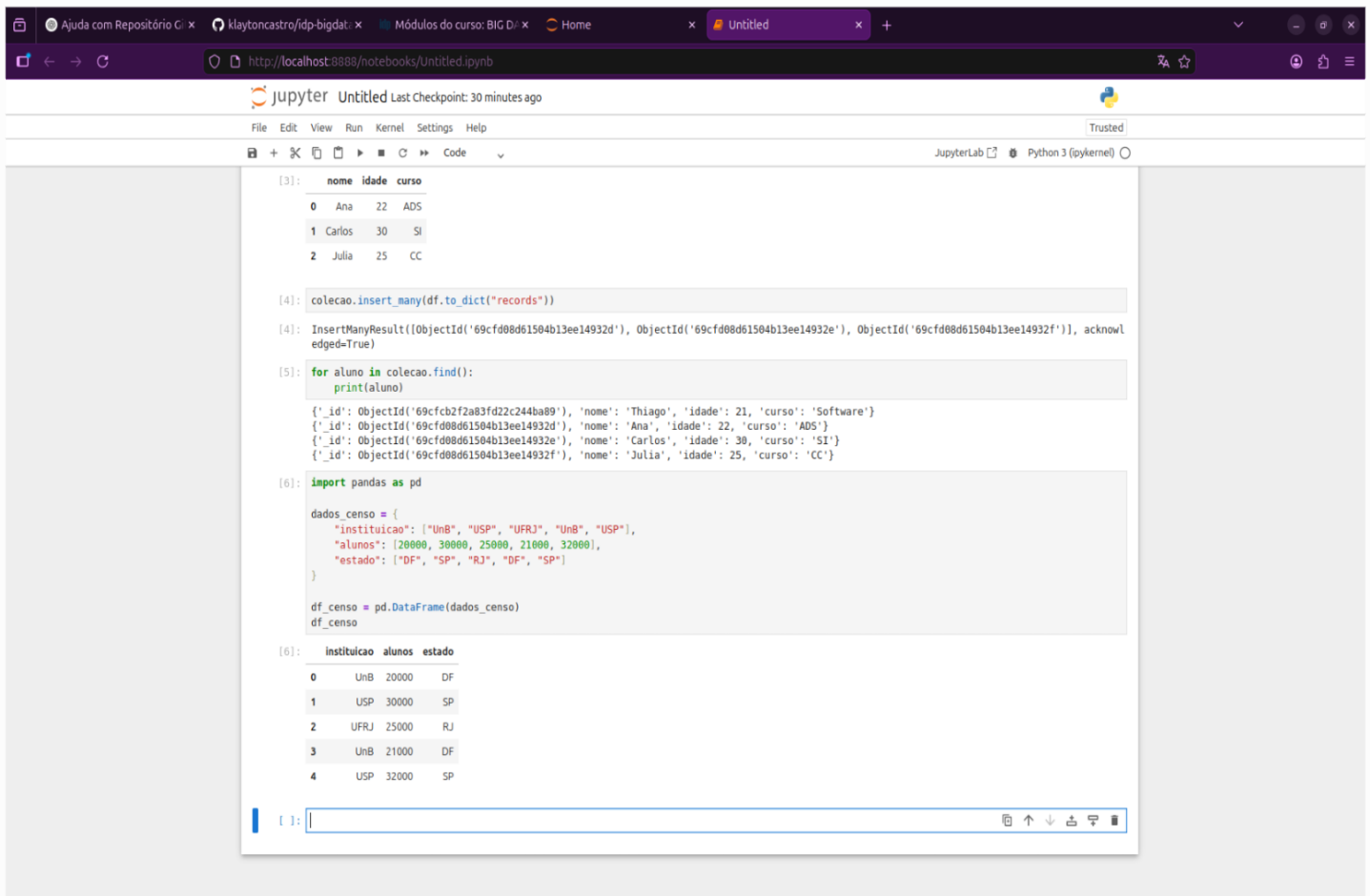
	nome	idade	curso
0	Ana	22	ADS
1	Carlos	30	SI
2	Julia	25	CC

```
[4]: colecacao.insert_many(df.to_dict("records"))

[4]: InsertManyResult([ObjectId('69cf08d61504b13ee14932d'), ObjectId('69cf08d61504b13ee14932e'), ObjectId('69cf08d61504b13ee14932f')], acknowledged=True)
```

The fourth cell is currently empty.

Figura 5 – Criação do DataFrame e inserção no MongoDB



```
[3]: nome idade curso
0 Ana 22 ADS
1 Carlos 30 SI
2 Julia 25 CC

[4]: colecao.insert_many(df.to_dict("records"))

[4]: InsertManyResult([ObjectId('69cf08d61504b13ee14932d'), ObjectId('69cf08d61504b13ee14932e'), ObjectId('69cf08d61504b13ee14932f')], acknowledged=True)

[5]: for aluno in colecao.find():
    print(aluno)

{'_id': ObjectId('69cf08d61504b13ee14932d'), 'nome': 'Thiago', 'idade': 21, 'curso': 'Software'}
{'_id': ObjectId('69cf08d61504b13ee14932e'), 'nome': 'Ana', 'idade': 22, 'curso': 'ADS'}
{'_id': ObjectId('69cf08d61504b13ee14932e'), 'nome': 'Carlos', 'idade': 30, 'curso': 'SI'}
{'_id': ObjectId('69cf08d61504b13ee14932f'), 'nome': 'Julia', 'idade': 25, 'curso': 'CC'}

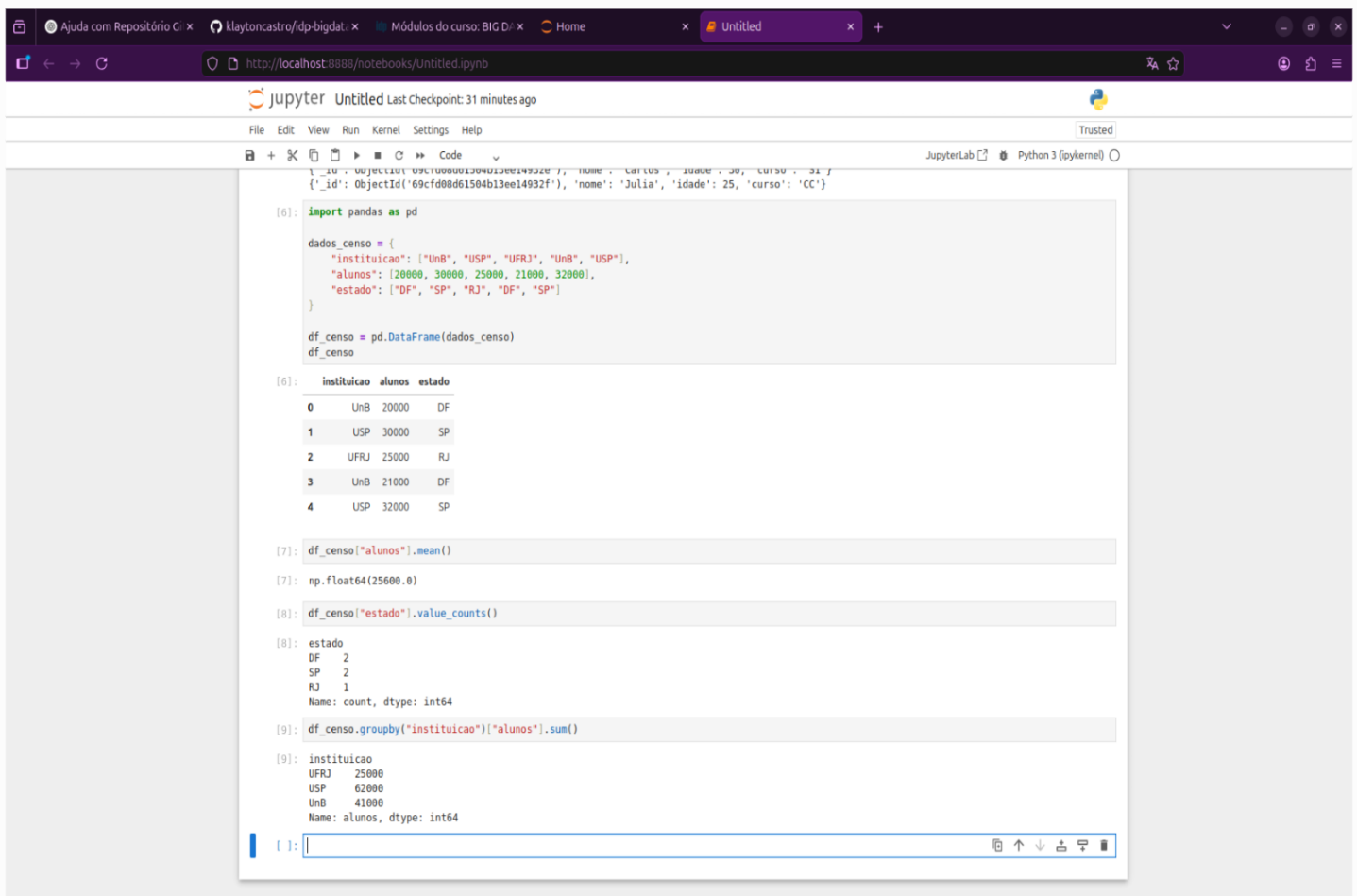
[6]: import pandas as pd

dados_censo = {
    "instituicao": ["UnB", "USP", "UFRJ", "UnB", "USP"],
    "alunos": [20000, 30000, 25000, 21000, 32000],
    "estado": ["DF", "SP", "RJ", "DF", "SP"]
}

df_censo = pd.DataFrame(dados_censo)
df_censo

[6]: instituicao alunos estado
0 UnB 20000 DF
1 USP 30000 SP
2 UFRJ 25000 RJ
3 UnB 21000 DF
4 USP 32000 SP
```

Figura 6 – Consulta e validação dos dados no MongoDB



```
[6]: import pandas as pd

dados_censo = {
    "instituicao": ["UnB", "USP", "UFRJ", "UnB", "USP"],
    "alunos": [20000, 30000, 25000, 21000, 32000],
    "estado": ["DF", "SP", "RJ", "DF", "SP"]
}

df_censo = pd.DataFrame(dados_censo)
df_censo

[6]: instituicao alunos estado
0 UnB 20000 DF
1 USP 30000 SP
2 UFRJ 25000 RJ
3 UnB 21000 DF
4 USP 32000 SP

[7]: df_censo["alunos"].mean()

[7]: np.float64(25600.0)

[8]: df_censo["estado"].value_counts()

[8]: estado
DF 2
SP 2
RJ 1
Name: count, dtype: int64

[9]: df_censo.groupby("instituicao")["alunos"].sum()

[9]: instituicao
UFRJ 25000
USP 62000
UnB 41000
Name: alunos, dtype: int64
```

Figura 7 – Análise de dados com Pandas